SYSTEM DESIGN TRAINING PROGRAM

Day 3

Scalability, Performance & Resilience

Case Study: RetailOrderHub — scaling and hardening the refactored monolith

Language: Java **Platform:** Azure Load Balancer / App Gateway, Azure Cache for Redis

Today's Agenda

Four hours, from single server to resilient at scale

01	**Recap & Bridge from Day 2**	From a clean refactor to a system under load
02	**Scaling & High Availability**	Horizontal vs vertical scaling, load balancing, failover
03	**Caching, CDN & Performance**	Speeding up RetailOrderHub, and measuring it
04	**Resilience & Circuit Breaker**	Retry, Timeout, Bulkhead, Fallback — and the Circuit Breaker pattern
05	**Hands-On Labs, Woven Throughout**	Each lab runs right after its topic — not saved for the end

Materials & Setup Checklist

What you'll need on hand before we start the labs

RetailOrderHub Codebase

The post-Day-2 refactored project, with PaymentService's call point ready to wrap with a Circuit Breaker

Azure Subscription Access

Your own subscription or free trial — you'll provision the Load Balancer and VMs yourself, live, via today's Azure Setup Guide

A Few Minutes to Add Resilience4j

You'll add this dependency yourself in Lab 3, along with a payment strategy you write to simulate a gateway failure on demand

Yesterday

We Fixed the Structure

From the SOLID refactor

- OrderManager became OrderService, InventoryService, and PaymentService
- Payment methods now extend via a PaymentStrategy interface, not if/else
- Every major dependency points at an interface, not a concrete class

Today, We Ask a Harder Question

A clean design doesn't automatically survive real traffic — what happens when 10,000 customers hit RetailOrderHub during a flash sale?

And what happens at 2am when the payment provider itself goes down — does one slow dependency take the whole system with it?

Yesterday's clean PaymentStrategy interface is exactly what makes today's Circuit Breaker possible — we can wrap the interface without touching PaymentService itself

Scale from Zero to Millions of Users

Horizontal vs. vertical scaling — techniques and trade-offs

Vertical Scaling

- Add more CPU, RAM, or disk to a single server
- Simple — no application changes needed
- Hard ceiling — there's a biggest machine you can buy
- Single point of failure remains

Horizontal Scaling

- Add more instances of the application server
- Effectively unlimited ceiling — just add more machines
- Removes the single point of failure
- Requires a load balancer and often application changes (statelessness)

RetailOrderHub today runs on a single instance — this is where we start scaling it out.

Load Balancers, Forward Proxy & Reverse Proxy

Two proxy patterns, and where a load balancer fits between them

Forward Proxy

- Sits in front of clients, forwarding their requests out to the internet
- Hides the client's identity from the destination server
- Typical uses: corporate internet gateways, VPNs, content filtering
- The destination server only ever sees the proxy, never the real client

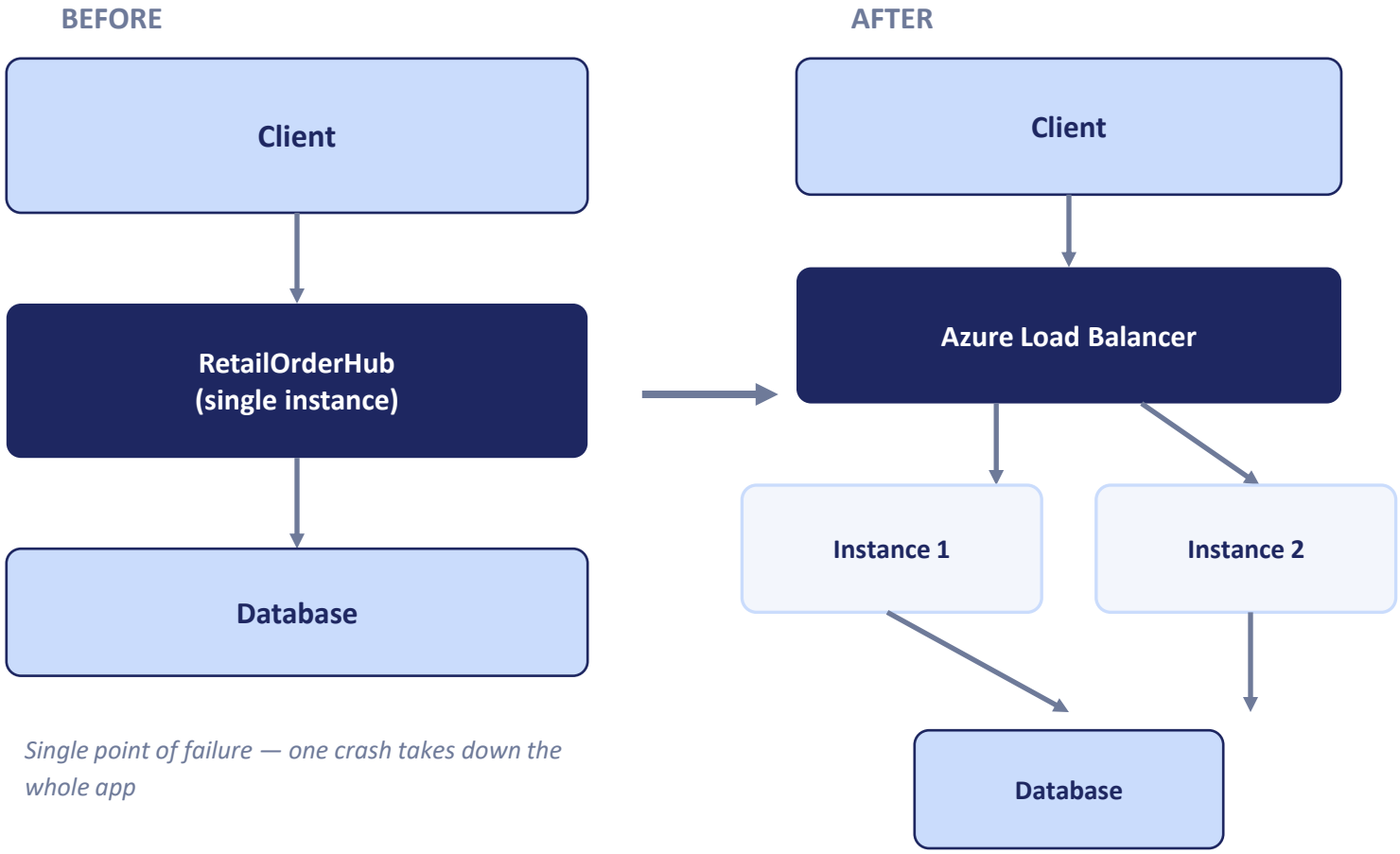
Reverse Proxy

- Sits in front of servers, accepting requests on their behalf
- Hides server identity and topology from the client
- Typical uses: load balancing, SSL termination, caching, WAF
- A Load Balancer is a specialized reverse proxy that also distributes traffic

Azure Load Balancer (Layer 4) and Application Gateway (Layer 7) are both reverse proxies in front of RetailOrderHub — App Gateway adds SSL termination, URL routing, and a Web Application Firewall.

Walkthrough: Scaling RetailOrderHub

From a single instance to a load-balanced, highly available web tier



Single point of failure — one crash takes down the whole app

Failover in action: If Instance 1 goes offline, the load balancer's health checks detect it and route all traffic to Instance 2 — no downtime for the customer.

This is exactly what today's Load Balancing lab walks through, live.

HANDS-ON LAB 1

Lab 1 — Scaling Whiteboard

Whiteboard exercise — evolve the architecture on paper before touching Azure

- Evolve RetailOrderHub from a single instance to a load-balanced, horizontally-scaled design
- Work on paper first — no Azure resources needed for this one
- Identify what has to change in the app itself to support multiple instances (statelessness)

High Availability

Why a single server or single database fails under load or outage

Health Checks

The load balancer continuously pings each instance; an unhealthy instance is pulled from rotation automatically.

Failover

Traffic is rerouted to healthy instances the moment a failure is detected — no manual intervention needed.

Auto-Scaling

New instances are added automatically as load increases, and removed as it drops, keeping cost proportional to demand.

Now the web tier looks good — what about the data tier? A single database still has no failover or redundancy. Database replication addresses that, but is out of scope for today — we'll pick it up on Day 4.

HANDS-ON LAB 2

Lab 2 — Load Balancing & Failover

Live failover demo first, then configure your own Azure Load Balancer

- Watch a live failover demo against a running two-instance setup
- Configure your own Azure Load Balancer in front of two RetailOrderHub instances
- Stop one instance yourself and measure your own failover, live

Caching and CDN

Types, expiration policies, and best practices

Caching

In-memory cache

Fastest — lives inside the app instance, but not shared across instances

Distributed cache

Shared across all instances (e.g. Azure Cache for Redis) — the right choice once you're horizontally scaled

Expiration policy

TTL (time-based) or LRU (least-recently-used) — stale data is a real risk, choose deliberately

Content Delivery Network (CDN)

- Caches static assets (images, CSS, JS) at edge locations close to the user
- Reduces load on the origin server and cuts latency dramatically for geographically distant users
- Consideration: cache invalidation — how do you push out an update to a cached asset quickly when needed?

Caching Strategies — An Overview

Who talks to the cache, and who keeps it in sync — five patterns, five trade-offs

Every caching strategy answers the same two questions differently: who reads from the cache, and who writes to it — the application, or the cache layer itself. Getting this wrong is how caches serve stale data or silently lose writes.

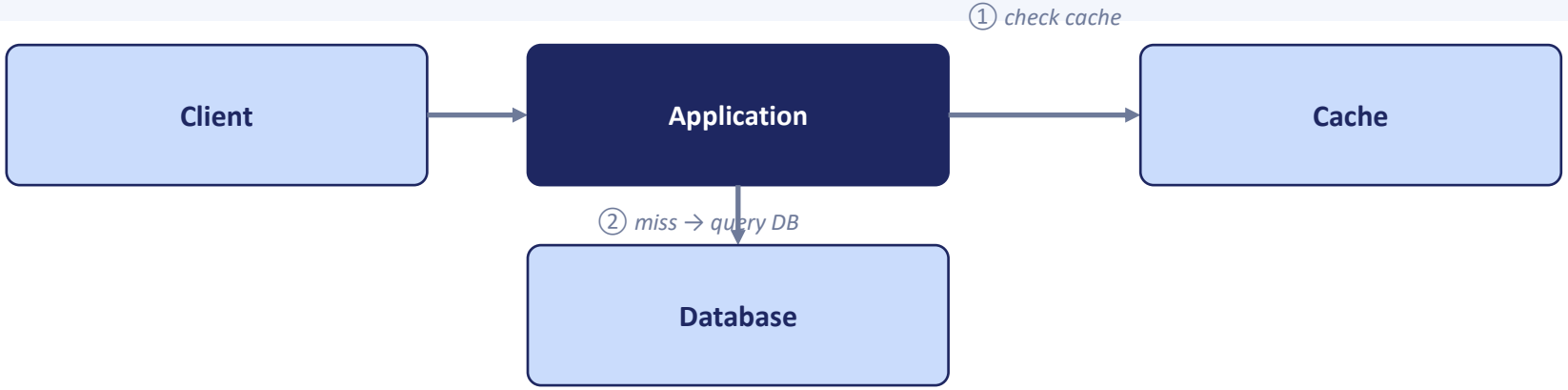
- **Cache-Aside (Lazy Loading) —** the application checks the cache first and loads on a miss itself
- **Read-Through —** the cache sits in front of the database and loads missing data on its own
- **Write-Through —** every write goes to the cache first, which writes through to the database immediately
- **Write-Behind (Write-Back) —** writes land in the cache and are flushed to the database asynchronously, later
- **Write-Around —** writes go straight to the database, bypassing the cache entirely

The next five slides walk through each pattern individually — *how it works, when to reach for it, and where it can bite you.*

Cache-Aside (Lazy Loading)

The application owns the cache — it decides what to load, and when

The application checks the cache first. On a miss, it queries the database itself, then writes the result back to the cache for next time. Simple, resilient, and the most common pattern in practice.



How It Works

① Check the cache. ② On a miss, query the database. ③ Write the result back to the cache — so the next request is a hit.

Best For:

Read-heavy workloads where the app already owns data-access code — product catalogs, user profiles, anything read far more than it's written.

Watch Out For:

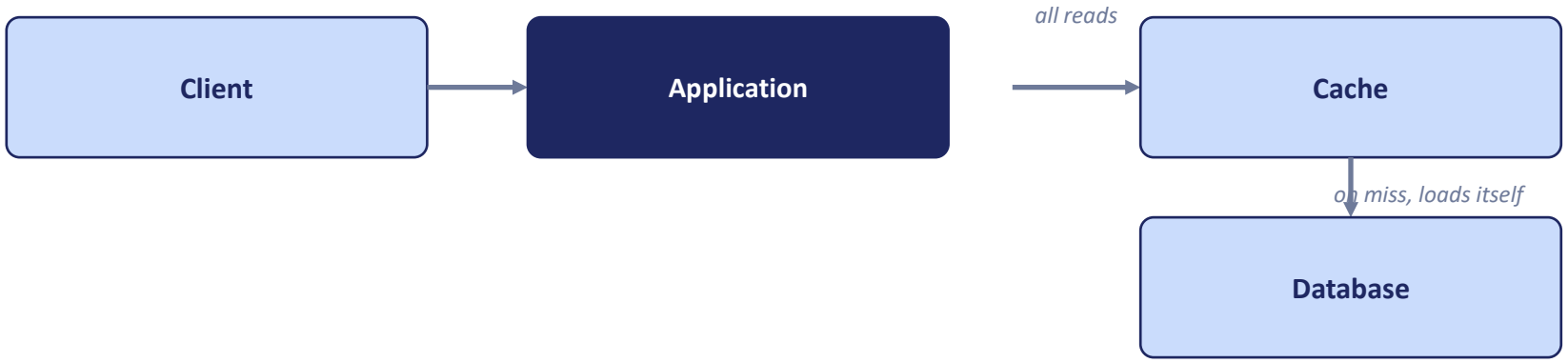
Every miss pays full database latency, and a stampede of simultaneous misses — e.g. right after a TTL expiry — can hit the database hard all at once.

Cache-Aside is the default choice when in doubt — *this is the pattern RetailOrderHub's catalog cache uses, applied concretely a little later in this section.*

Read-Through

The cache mediates every read — the application never talks to the database directly

The cache sits directly in front of the database. On a miss, the cache loads the missing data itself, then returns it — the application only ever talks to the cache, never the database.



How It Works

The application only ever calls the cache. On a miss, the cache queries the database itself, stores the result, and returns it.

Best For:

Read-heavy systems where you want caching logic centralized in the cache layer, instead of duplicated across every service that reads the data.

Watch Out For:

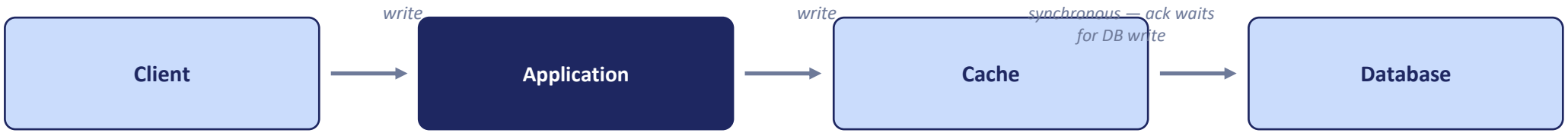
Ties you to a caching layer that actually supports read-through loading — not every cache library or managed cache offers this out of the box.

***RetailOrderHub's catalog cache uses Cache-Aside instead** — Azure Cache for Redis needs the application to own the load-on-miss logic, so Cache-Aside is the closer fit there.*

Write-Through

Cache and database are never out of sync — at the cost of write latency

Every write goes to the cache first. The cache immediately writes it through to the database, and the write isn't acknowledged until the database confirms it. Cache and database stay in sync at all times.



Best For:

Data where staleness is unacceptable — pricing, inventory counts, anything downstream systems must see updated immediately after a write.

Watch Out For:

Every write now pays the full cost of a database write plus a cache write — there's no latency win on the write path, only on reads afterward.

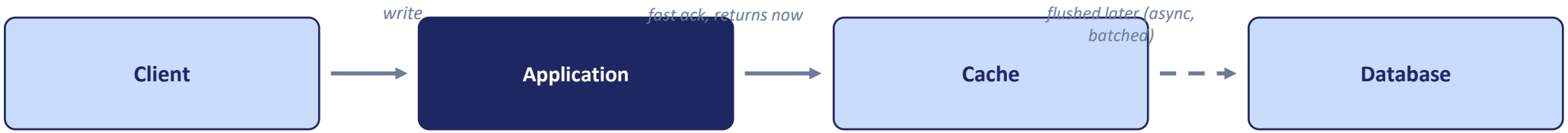
If RetailOrderHub moved product stock levels to Write-Through, *a flash-sale checkout would never oversell stock — at the cost of a slightly slower checkout write.*

Contrast with Cache-Aside: Cache-Aside only updates the cache on a read miss; Write-Through updates it proactively on every write, so a read right after a write is always a hit.

Write-Behind (Write-Back)

Very fast writes — with a real risk if the cache fails before it flushes

Writes land in the cache and are acknowledged immediately. The cache flushes them to the database asynchronously, later — often batched together with other writes for efficiency.



Best For:

Very high write volume where write latency matters more than a small risk of losing the most recent writes — metrics, counters, activity logs.

Watch Out For:

A cache failure before the flush completes loses those writes permanently — never use this for data RetailOrderHub can't afford to lose.

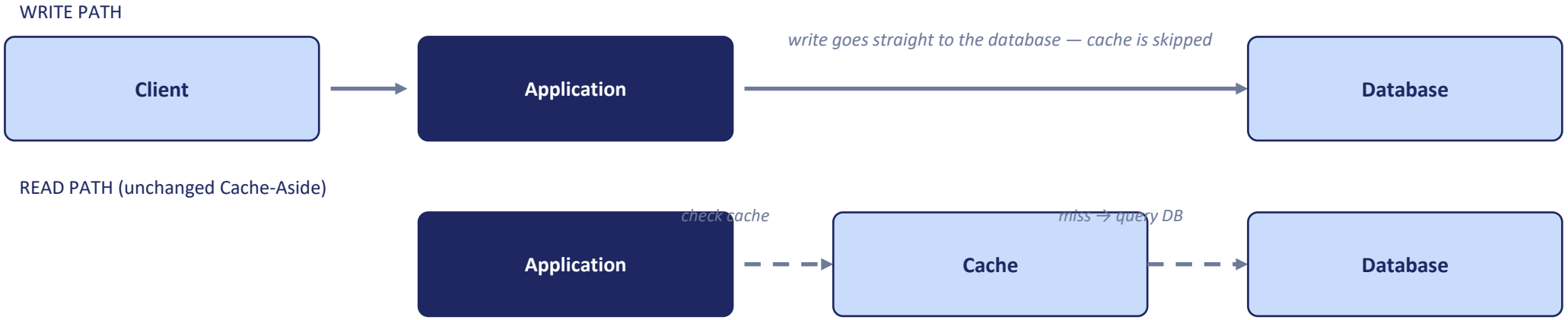
Write-Behind is a poor fit for PaymentService — *a lost write there means a customer paid and the system doesn't know it. Reserve Write-Behind for data that's safe to lose.*

Where it could help RetailOrderHub: logging catalog page views or search queries for analytics — fast writes, and losing a few events on a rare cache crash is a non-issue.

Write-Around

Writes bypass the cache entirely — reads still follow Cache-Aside

Writes go straight to the database, bypassing the cache entirely. Reads still follow Cache-Aside — checking the cache first, and loading from the database on a miss.



Best For:

Write-heavy data that's rarely read right after being written — bulk imports, audit logs, historical records nobody queries right away.

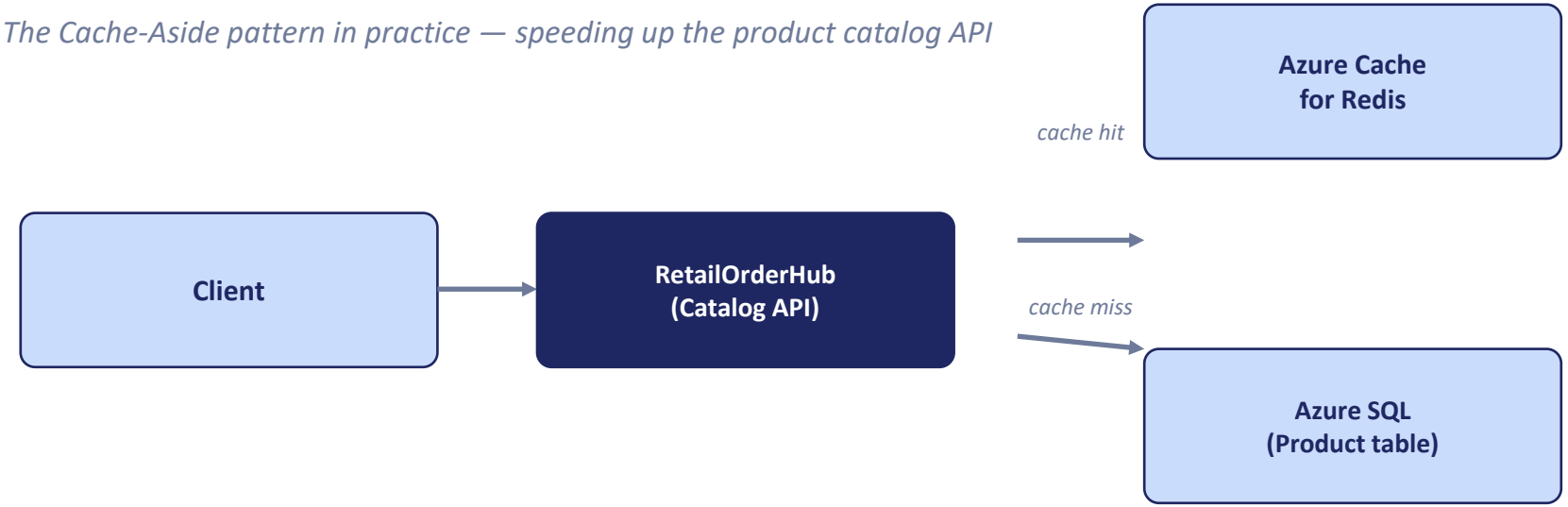
Watch Out For:

A read immediately after a write is guaranteed to miss the cache and pay full database latency — freshly written data is never pre-warmed.

A bulk price-list import into RetailOrderHub is a good Write-Around candidate — *the catalog cache doesn't need those rows warmed until someone actually browses the updated prices.*

Applying Caching to RetailOrderHub

The Cache-Aside pattern in practice — speeding up the product catalog API



Cache Hit vs. Miss

Hit: Redis has the data — return in ~1ms, skip the database entirely.

Miss: not in Redis — query Azure SQL, then populate Redis for next time.

This is the Cache-Aside pattern from a few slides back, applied concretely: RetailOrderHub owns the check-cache, query-on-miss, populate-cache logic itself.

Expiration policy for RetailOrderHub's catalog: a short TTL (e.g. 60 seconds) balances freshness against load — product prices and stock levels shouldn't be stale for long.

Cache Expiration & Invalidation

A cache that never forgets is just a slow, silent way to serve wrong data

Every cached item needs a way to leave the cache. The question isn't whether to expire data — it's which policy decides when, and how well that policy matches how the data actually changes.

TTL — Time to Live

Every entry expires automatically after a fixed duration, regardless of use. Simple to reason about, and the default choice for most caches — pick a duration that matches how fast the underlying data actually changes.

LRU — Least Recently Used

When the cache is full, evict whichever entry hasn't been read in the longest time. Keeps "hot" data in the cache automatically — the right default when memory, not staleness, is the constraint.

LFU — Least Frequently Used

Evict whichever entry has been read the fewest times, not just the least recently. Resists one-off bursts pushing out data that's normally popular — more bookkeeping overhead than LRU.

Event-Based Invalidation

The application explicitly removes or updates a cache entry the moment the underlying data changes — a write, an admin edit, a webhook. The freshest option, but only as reliable as every code path that's supposed to trigger it.

Policies stack: *TTL and LRU/LFU are not mutually exclusive — a cache commonly uses a TTL as a safety net and an eviction policy for memory pressure at the same time.*

Cache Expiration: Recommended Practices

Stale data and empty caches are both failure modes — plan for both

There's no universal "right" TTL. The right expiration strategy depends on how often the underlying data changes, and how expensive it is to be wrong — set it deliberately, per data type, not once for the whole cache.

- **Match TTL to volatility, per key —** product prices and stock levels need a short TTL (seconds); a category list or store-hours page can safely live for hours.
- **Invalidate on write when it matters —** for data where staleness is costly, pair a TTL safety net with explicit invalidation the moment the underlying record changes.
- **Guard against cache stampede —** when a hot key expires, many simultaneous requests can all miss at once and hammer the database; stagger TTLs slightly or use a lock so only one request repopulates it.
- **Warm the cache before load hits it —** for predictable spikes like a flash sale, pre-populate the catalog cache ahead of time instead of letting the first wave of customers pay the miss cost.
- **Monitor hit rate, not just uptime —** a cache with a low hit rate is barely helping and may be worth re-tuning; track it per key pattern, not just as one deck-wide average.

RetailOrderHub's catalog cache: *a short TTL (60 seconds) balances freshness against database load — the same TTL we saw a couple of slides back, applied to the catalog API.*

Performance Optimization

Profiling, monitoring, and planning for growth

Profiling & Monitoring

Measure before you optimize. Track response time, throughput, and error rate per endpoint — you can't fix what you don't measure.

Optimization Techniques

Reduce unnecessary database round-trips, batch queries where possible, and cache expensive computations — the catalog cache from a moment ago is one example.

Capacity Planning

Forecast growth using historical traffic patterns, and provision ahead of demand spikes — not in reaction to an outage.

Rule of thumb: measure, then optimize the biggest bottleneck — not the one that's most interesting to fix.

Resilience Patterns

Partial failures are normal in distributed systems, not exceptional

Retry

Automatically re-attempt a failed call — useful for transient failures, dangerous without a backoff strategy

Timeout

Never wait forever for a response — a slow dependency shouldn't be allowed to hang your entire request

Bulkhead

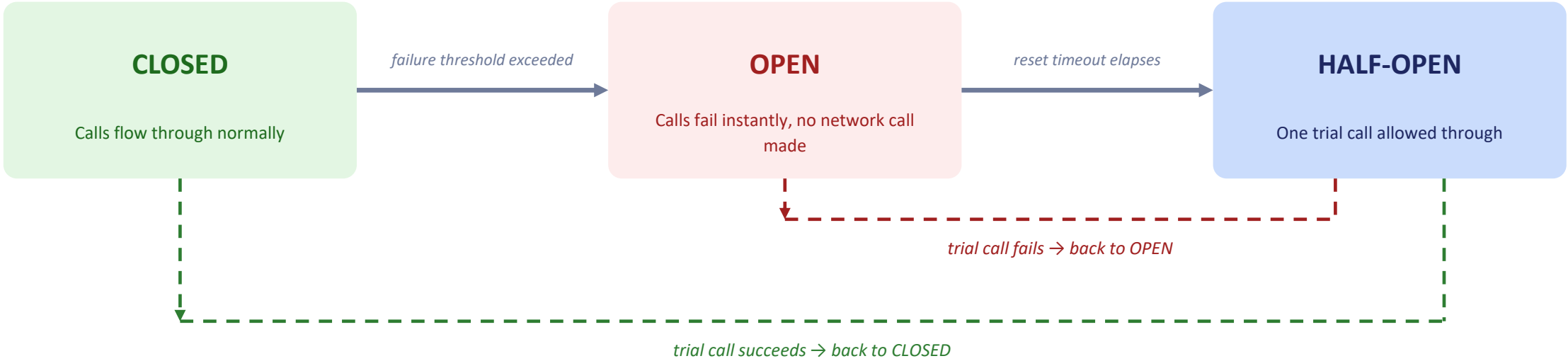
Isolate resources per dependency, so one overloaded dependency can't starve every other request of threads or connections

Fallback

When a call fails, degrade gracefully — return cached or default data instead of an error page

The Circuit Breaker Pattern

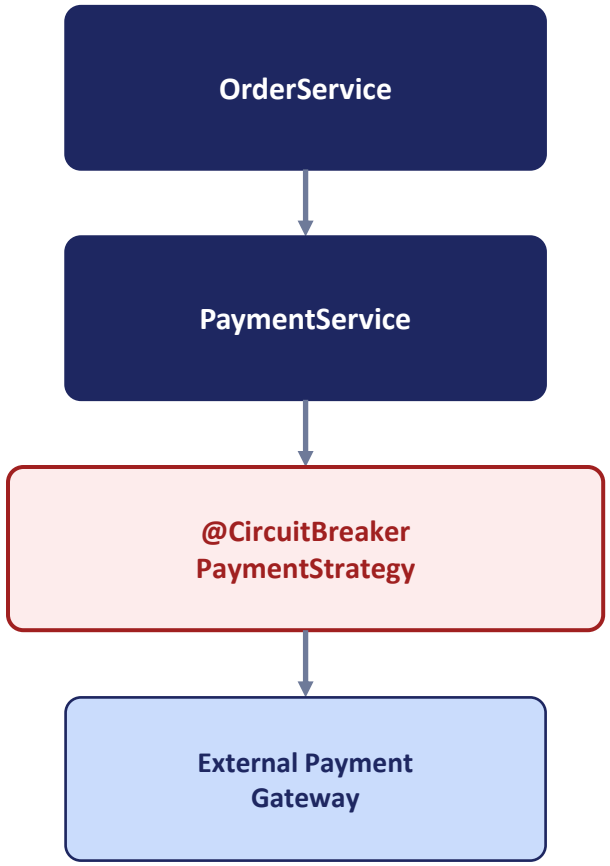
When to stop calling a failing dependency, and when to try again



Why not just retry forever? If a dependency is genuinely down, retrying every request just adds load to something already struggling, and makes your own service slow too. The Circuit Breaker fails fast instead — protecting both sides.

Applying the Circuit Breaker to PaymentService

Yesterday's clean interface makes today's resilience simple



What Resilience4j Adds — No Changes to PaymentService

- The @CircuitBreaker annotation wraps calls to PaymentStrategy — a decorator around the interface, not a rewrite of it
- This only works cleanly because PaymentService already depends on the PaymentStrategy interface, not a concrete class — yesterday's DIP work pays off today
- When the gateway starts failing repeatedly, the breaker trips to Open and PaymentService gets a fast, predictable failure instead of a long hang
- A fallback method returns a clear "payment temporarily unavailable" response — better than a generic 500 error

HANDS-ON LAB 3

Lab 3 — Circuit Breaker Lab

Wrap PaymentService with Resilience4j and watch it trip live

- Wrap PaymentService's calls with a Resilience4j Circuit Breaker
- Simulate a payment gateway failure on demand
- Watch the breaker trip to Open, then recover through Half-Open back to Closed

Day 3 Wrap-Up

What we covered — and where it's going

Key Takeaways

- RetailOrderHub now runs load-balanced across multiple instances
- The product catalog is cached in Redis, cutting response time dramatically
- PaymentService fails fast instead of hanging when the gateway is down
- Yesterday's DIP work is what made today's Circuit Breaker a clean addition

Expectation Coverage — Day 3

✓ **Scaling application**

✓ **Resilience**

✓ **Circuit Breaker**

Looking ahead — Day 4: We optimize and shard RetailOrderHub's database, and begin decomposing the monolith into OrderService, InventoryService, and PaymentService microservices.